ALGORITHMIC COMPLEXITY IN SOFTWARE DEVELOPMENT

WHAT IS ALGORITHMIC COMPLEXITY?

Algorithmic complexity measures the amount of resources an algorithm needs to run.

The most important resources are:

⏱ Time: how long it takes to run.

💾 Space: how much memory it uses.

Its purpose is to determine how efficient a solution is as the amount of data processed increases.

WHY IS IT IMPORTANT?

- ✅ Improves application performance.
- ✅ Reduces response times.
- ✅ Optimizes memory usage.
- ✅ Enables the development of scalable software.
- ✅ Helps compare different algorithms.

📈 BIG O NOTATION

Big O notation describes how the running time of an algorithm grows as the size of the data (n) increases.

MOST COMMON COMPLEXITIES

Complexity	Name	Example
O(1)	Constant	Accessing an element of an array
O(log n)	Logarithmic	Binary search
O(n)	Linear	a for loop
O(n log n)	Linear logarithmic	Merge Sort
O(n²)	Quadratic	Nested loops
O(2^n)	Exponential	Complex recursive problems
O(n!)	Factorial	Traveling Salesman Problem

📊 EFFICIENCY COMPARISON

🥇 O(1) = Most efficient

🥈 O(log n)

🥉 O(n)

⚠️ O(n log n)

❌ O(n²)

❌ O(2^n)

🚫 O(n!)

The faster the complexity increases, the worse the performance becomes for large amounts of data.

🔍 TEMPORAL COMPLEXITY

Measures the number of operations an algorithm performs.

```
for (i = 0; i < n; i++){
    printf(i);
}
```

It executes n times.

Complexity:

O(n)

💾 SPATIAL COMPLEXITY

It measures the additional memory required by an algorithm.

Example: `int array[n];`

Memory grows proportionally to the size of the data.

Space complexity: O(n)

🛠 EXAMPLES IN SOFTWARE DEVELOPMENT

🔎 **Search Engines**
They use efficient algorithms to find information quickly.

🛒 **E-commerce**
Search optimization and product recommendations.

🎮 **Video Games**
Efficient management of characters, maps, and physics.

☁️ **Cloud Systems**
High-performance, massive data processing.

📌 GOOD PRACTICES

- ✓ Choose appropriate data structures.
- ✓ Avoid unnecessary loops.
- ✓ Reduce repetitive operations.
- ✓ Analyze data growth.
- ✓ Prioritize efficient algorithms.